

CRYPTOLOGY

Aditi Biju

261100690027

Lab 3:

Cryptography Lab — Introduction to Python

Python Basics and Debugging

1. Debug the following programs. Identify the type of error in each case.

```
print("Welcome to Cryptology Lab")  
  
message = "Python Programming"  
  
print(mesage)  
  
num = input("Enter a number: ")  
  
result = num + 10  
  
print(result)
```

```
print("Welcome to Cryptology Lab")  
message = "Python Programming"  
print(message)  
num = int(input("Enter a number: "))  
result = num + 10  
print(result)
```

```
(aduuu@kali)-[~]  
$ python3 q1.py  
Welcome to Cryptology Lab  
Python Programming  
Enter a number: 5  
15
```

2. Write Python statements to display the following exactly:

Welcome to Cryptology Lab

It's interesting to learn Python.

Students, "Welcome to the Cryptology Lab".

```
print("Welcome to Cryptology Lab")  
print("It's interesting to learn Python. ")  
print('students. "welcome to the Cryptology Lab".')
```

```
(aduuu@kali)-[~]  
$ python3 q2.py  
Welcome to Cryptology Lab  
It's interesting to learn Python.  
students. "welcome to the Cryptology Lab".
```

3. Read two numbers from the user and perform:

Addition

Subtraction

Multiplication

Division

Integer division

Remainder

Exponentiation

Display each result with an appropriate message.

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
print("Addition:", num1 + num2)
print("Subtraction:", num1 - num2)
print("Multiplication:", num1 * num2)
print("Division:", num1 / num2)
print("Integer Division:", num1 // num2)
print("Remainder:", num1 % num2)
print("Exponentiation:", num1 ** num2)
```

```
(aduuu@kali)-[~]
$ python3 q3.py
Enter first number: 4
Enter second number: 2
Addition: 6.0
Subtraction: 2.0
Multiplication: 8.0
Division: 2.0
Integer Division: 2.0
Remainder: 0.0
Exponentiation: 16.0
```

4. Read a floating-point number and display:

its data type;

its absolute value;

the value rounded to two decimal places.

```
num = float(input("Enter a floating-point number: "))
print("Data type:", type(num))
print("Absolute value:", abs(num))
print("Rounded value:", round(num, 2))
```

```
(aduuu@kali)-[~]
$ python3 q4.py
Enter a floating-point number: 12.134
Data type: <class 'float'>
Absolute value: 12.134
Rounded value: 12.13
```

Part B — Data Types and Collections (~25 min)

5. Create variables containing examples of:

int, float, complex, str, list, tuple, set, and dictionary.

Use Python to display the value and data type of each variable.

```
a = 10
b = 3.14
c = 2 + 3j
d = "Python"
e = [1, 2, 3]
f = (1, 2, 3)
g = {1, 2, 3}
h = {"name": "Aditi", "age": 20}
print("Value:", a, "Data type:", type(a))
print("Value:", b, "Data type:", type(b))
print("Value:", c, "Data type:", type(c))
print("Value:", d, "Data type:", type(d))
print("Value:", e, "Data type:", type(e))
print("Value:", f, "Data type:", type(f))
print("Value:", g, "Data type:", type(g))
print("Value:", h, "Data type:", type(h))
```

```
(aduuu@kali)-[~]
$ python3 q5.py
Value: 10 Data type: <class 'int'>
Value: 3.14 Data type: <class 'float'>
Value: (2+3j) Data type: <class 'complex'>
Value: Python Data type: <class 'str'>
Value: [1, 2, 3] Data type: <class 'list'>
Value: (1, 2, 3) Data type: <class 'tuple'>
Value: {1, 2, 3} Data type: <class 'set'>
Value: {'name': 'Aditi', 'age': 20} Data type: <class 'dict'>
```

6. Demonstrate experimentally that:

Strings are immutable, whereas lists are mutable.

Include the Python statements you used and explain the observed result.

```
message = "HELLO"
print("Original string:", message)
try:
    message[0] = "Y"
except TypeError:
    print("Strings are immutable and cannot be changed.")
numbers = [1, 2, 3]
print("Original list:", numbers)
numbers[0] = 10
print("Modified list:", numbers)
```

```
(aduuu@kali)-[~]
$ python3 q6.py
Original string: HELLO
Strings are immutable and cannot be changed.
Original list: [1, 2, 3]
Modified list: [10, 2, 3]
```

7. Create a list containing five prime numbers. Perform the following:

display the first three elements;

display the last element;

append another prime number;

extend the list with two more prime numbers;

display the total number of elements.

```
primes = [2, 3, 5, 7, 11]
print("First three elements:", primes[:3])
print("Last element:", primes[-1])
primes.append(13)
print("After appending:", primes)
primes.extend([17, 19])
print("After extending:", primes)
print("Total number of elements:", len(primes))
```

```
(aduuu@kali)-[~]
$ python3 q7.py
First three elements: [2, 3, 5]
Last element: 11
After appending: [2, 3, 5, 7, 11, 13]
After extending: [2, 3, 5, 7, 11, 13, 17, 19]
Total number of elements: 8
```

Part C — String Operations

For Questions 8–12, use:

message = "CRYPTOLOGY USING PYTHON"

8. Display:

length of the string;

first five characters;

last six characters;

characters from positions 3 to 8;

string in reverse order.

```
message = "CRYPTOLOGY USING PYTHON"
print("Length:", len(message))
print("First five characters:", message[:5])
print("Last six characters:", message[-6:])
print("Characters from positions 3 to 8:", message[3:9])
print("Reverse order:", message[::-1])
```

```
(aduuu@kali)-[~]
$ python3 q8.py
Length: 23
First five characters: CRYPT
Last six characters: PYTHON
Characters from positions 3 to 8: PTOLOG
Reverse order: NOHTYP GNISU YGOLOTPYRC
```

9. Perform the following:

convert to lowercase;

convert back to uppercase;

check whether "PYTHON" exists in the string;

replace "PYTHON" with "PROGRAMMING".

```
message = "CRYPTOLOGY USING PYTHON"
print("Lowercase:", message.lower())
print("Uppercase:", message.upper())
print("PYTHON exists:", "PYTHON" in message)
print("After replacement:", message.replace("PYTHON", "PROGRAMMING"))
```

```
(aduuu@kali)-[~]
$ python3 q9.py
Lowercase: cryptography using python
Uppercase: CRYPTOLOGY USING PYTHON
PYTHON exists: True
After replacement: CRYPTOLOGY USING PROGRAMMING
```

10. Remove the spaces from the string and display the resulting string.

```
message = "CRYPTOLOGY USING PYTHON"
result = message.replace(" ", "")
print("String without spaces:", result)
```

```
(aduuu@kali)-[~]
$ python3 q10.py
String without spaces: CRYPTOLOGYUSINGPYTHON
```

11. Using a loop, display every character and its corresponding Unicode/ASCII value using ord().

Example:

C -> 67

R -> 82

Y -> 89

...

```
message = "CRYPTOLOGY USING PYTHON"
for character in message:
    print(character, "→", ord(character))
```

```
(aduuu@kali)-[~]
$ python3 q11.py
C → 67
R → 82
Y → 89
P → 80
T → 84
O → 79
L → 76
O → 79
G → 71
Y → 89
→ 32
U → 85
S → 83
I → 73
N → 78
G → 71
→ 32
P → 80
Y → 89
T → 84
H → 72
O → 79
N → 78
```

12. Use chr() to convert the following values into characters:

65, 67, 73, 80, 72, 69, 82

Combine the resulting characters and identify the word.

```
values = [65, 67, 73, 80, 72, 69, 82]
word = ""
for value in values:
    word += chr(value)
print("Characters:", word)
print("word:", word)
```

```
(aduuu@kali)-[~]
$ python3 q12.py
Characters: ACIPHER
word: ACIPHER
```

Part D — Text Processing for Cryptology

13. Plaintext preprocessing

Read a message from the user and:

remove leading/trailing spaces;

convert it to uppercase;

remove spaces between words.

Example:

Input : meet me tomorrow

Output: MEETMETOMORROW

```
message = input("Enter a message: ")
message = message.strip()
message = message.upper()
message = message.replace(" ", "")
print("Output:", message)
```

```
(aduuu@kali)-[~]
$ python3 q13.py
Enter a message: hello world
Output: HELLOWORLD
```

14. Character grouping

Read a string and divide it into blocks of five characters.

Example:

Input: CRYPTOLOGYLAB

Output:

CRYPT

LOGY

LAB

```
message = input("Enter a string: ")
for i in range(0, len(message), 5):
    print(message[i:i + 5])
```

```
(aduuu@kali)-[~]
$ python3 q14.py
Enter a string: CRYPTPOLOGYLAB
CRYPTP
LOGY
LAB
```

15. Padding

Modify Question 14 so that an incomplete final block is padded with X.

Input: CRYPTOLOGYLAB

Output:

CRYPT

LOGY

LABXX

```
message = input("Enter a string: ")
while len(message) % 5:
    message += "X"
for i in range(0, len(message), 5):
    print(message[i:i+5])
```

```
(aduuu@kali)-[~]
$ python3 q15.py
Enter a string: CRYPTOLOGYLAB
CRYPT
LOGY
LABXX
```

16. Character frequency

For the following text:

```
text = "CRYPTOLOGYISINTERESTING"
```

write a program to calculate the frequency of each character.

Then identify the most frequently occurring character.

```
text = "CRYPTOLOGYISINTERESTING"
frequency = {}
for character in text:
    if character in frequency:
        frequency[character] += 1
    else:
        frequency[character] = 1
for character, count in frequency.items():
    print(character, ":", count)
most_frequent = max(frequency, key=frequency.get)
print("Most frequently occurring character:", most_frequent)
print("Frequency:", frequency[most_frequent])
```

```
(aduuu@kali)~$ python3 q16.py
C : 1
R : 2
Y : 2
P : 1
T : 3
O : 2
L : 1
G : 2
I : 3
S : 2
N : 2
E : 2
Most frequently occurring character: T
Frequency: 3
```

17. Percentage frequency

Extend Question 16 to calculate:

Percentage Frequency=

Total Number of Characters/ Frequency of Character $\times 100$

Display the percentage rounded to two decimal places.

```
text = "CRYPTOLOGYISINTERESTING"
frequency = {}
for character in text:
    if character in frequency:
        frequency[character] += 1
    else:
        frequency[character] = 1
total = len(text)
for character, count in frequency.items():
    percentage = (count / total) * 100
    print(character, ":", round(percentage, 2), "%")
```

```
(aduuu@kali)~$ python3 q17.py
C : 4.35 %
R : 8.7 %
Y : 8.7 %
P : 4.35 %
T : 13.04 %
O : 8.7 %
L : 4.35 %
G : 8.7 %
I : 13.04 %
S : 8.7 %
N : 8.7 %
E : 8.7 %
```

Part E — Mathematical Foundations for Cryptology

18. Modular arithmetic

Use Python to evaluate:

$29 \bmod 26$

$55 \bmod 26$

$78 \bmod 26$

$-3 \bmod 26$

$-29 \bmod 26$

Then write a program that accepts an integer and calculates the integer modulo 26.

```
print("29 mod 26 =", 29 % 26)
print("55 mod 26 =", 55 % 26)
print("78 mod 26 =", 78 % 26)
print("-3 mod 26 =", -3 % 26)
print("-29 mod 26 =", -29 % 26)
num = int(input("Enter an integer: "))
print("Modulo 26:", num % 26)
```

```
(aduuu@kali)-[~]
$ python3 q18.py
29 mod 26 = 3
55 mod 26 = 3
78 mod 26 = 0
-3 mod 26 = 23
-29 mod 26 = 23
Enter an integer: 5
Modulo 26: 5
```

19. Alphabet representation

Using

$A=0, B=1, \dots, Z=25$

write a program that accepts an uppercase character and displays its numerical representation.

Example:

Enter character: M

M -> 12

Also perform the reverse conversion:

Enter number: 12

12 -> M

```
character = input("Enter character: ")
number = ord(character) - ord('A')
print(character, "→", number)
number = int(input("Enter number: "))
character = chr(number + ord('A'))
print(number, "→", character)
```

```
(aduuu@kali)-[~]
$ python3 q19.py
Enter character: G
G → 6
Enter number: 7
7 → H
```

20. Prime numbers

Write a program to:

determine whether a given number is prime;

display all prime numbers within a range specified by the user.

```
num = int(input("Enter a number: "))
count = 0
for i in range(1, num + 1):
    if num % i == 0:
        count += 1
if count == 2:
    print(num, "is a prime number")
else:
    print(num, "is not a prime number")

start = int(input("Enter starting number: "))
end = int(input("Enter ending number: "))
print("Prime numbers:")
for num in range(start, end + 1):
    count = 0
    for i in range(1, num + 1):
        if num % i == 0:
            count += 1
    if count == 2:
        print(num, end=" ")
```

```
(aduuu@kali)-[~]
$ python3 q20.py
Enter a number: 23
23 is a prime number
Enter starting number: 1
Enter ending number: 23
Prime numbers:
2 3 5 7 11 13 17 19 23
```

21. GCD and coprimes

Read two integers from the user. Find their GCD and determine whether they are coprime.

Test your program with:

15 and 26

18 and 24

17 and 26

```
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

a = num1
b = num2
while b:
    a, b = b, a % b

gcd = a
print("GCD:", gcd)

if gcd == 1:
    print("The numbers are coprime")
else:
    print("The numbers are not coprime")
```

```
(aduuu@kali)-[~]
$ python3 q21.py
Enter first number: 37
Enter second number: 23
GCD: 1
The numbers are coprime
```

22. Write a program to get the multiplicative inverse of an integer b.

```
b = int(input("Enter b: "))
inverse = 0
for x in range(1, 26):
    if (b * x) % 26 == 1:
        inverse = x
        break
if inverse:
    print("Multiplicative inverse:", inverse)
else:
    print("Multiplicative inverse does not exist")
```

```
(aduuu@kali)-[~]
$ python3 q22.py
Enter b: 7
Multiplicative inverse: 15
```

23. WAP in python to implement client and server communication.

```
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 5000))
server.listen(1)
print("Server waiting for connection...")
connection, address = server.accept()
print("Client connected.")
message = connection.recv(1024).decode()
print("Message from client:", message)
connection.send("Hello from Server".encode())
connection.close()
server.close()
```

```
(aduuu@kali)-[~]
$ python3 server.py
Server waiting for connection...
Client connected.
Message from client: hellooo server
```

```
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 5000))
message = input("Enter message: ")
client.send(message.encode())
response = client.recv(1024).decode()
print("Response from server:", response)
client.close()
```

```
(aduuu@kali)-[~]
$ python client.py
Enter message: hellooo server
Response from server: Hello from Server
```